

Aula 04

*DataPrev (Perfil 3: Desenvolvimento de Software) Engenharia de Software - 2026
(Pós-Edital)*

Autor:

**Diego Carvalho, Equipe
Informática 2 (Diego Carvalho),
Equipe Informática 1 (Diego
Carvalho)**

09 de Julho de 2026

Índice

1) Kanban	3
2) Kanban - Questões Comentadas - FGV	8
3) Kanban - Lista de Questões - FGV	16
4) TDD	22
5) Questões Comentadas - TDD - Multibancas	30
6) Lista de Questões - TDD - Multibancas	41

KANBAN

Conceitos Básicos

INCIDÊNCIA EM PROVA: ALTA

O Kanban é um método de gerenciamento visual e melhoria de fluxo de trabalho. Diferente de uma metodologia ágil no sentido estrito, ele pode ser aplicado em diversos contextos, não se limitando apenas ao desenvolvimento de software. Seu foco principal é no fluxo do trabalho, e não no desempenho individual dos membros da equipe.

O Kanban busca a entrega contínua e sustentável de valor, evitando a sobrecarga da equipe. Ele tem sua origem no sistema de produção da Toyota, onde foi desenvolvido para otimizar a produção e reduzir desperdícios. Posteriormente, foi adaptado para o desenvolvimento de software e outras áreas, mantendo seus princípios fundamentais de visualização e gerenciamento de fluxo.

Esse método oferece visualização clara do fluxo de trabalho e uso de limites de WIP, ajudando a reduzir gargalos e aumentar a previsibilidade, ideal para lidar com atrasos e falta de visibilidade. Ele aumenta a transparência e a colaboração entre membros, o que favorece o engajamento e melhora o clima organizacional.

O Kanban aumenta a produtividade e a eficiência ao limitar o trabalho em progresso (WIP), evitando sobrecarga e melhorando o fluxo. O Kanban não isola a equipe – ele favorece a transparência e colaboração. O plano de sustentabilidade busca equilibrar demanda e capacidade, melhorando desempenho e satisfação.

Princípios e Práticas

INCIDÊNCIA EM PROVA: ALTÍSSIMA

O Kanban se baseia em um conjunto de princípios e práticas que visam otimizar o fluxo de trabalho e a entrega de valor. São eles:

Visualização do Fluxo de Trabalho

Uma das práticas centrais do Kanban é a visualização do fluxo de trabalho, geralmente por meio de um quadro Kanban. Este quadro é composto por colunas que representam as etapas do processo, como "A fazer", "Fazendo" e "Feito". Essa visualização clara do fluxo ajuda a identificar gargalos e obstáculos entre as tarefas, aumentando a transparência e a colaboração entre os membros da equipe. O acompanhamento visual com o quadro é uma característica central do Kanban.

Limitação do Trabalho em Progresso (WIP)

Limitar o WIP (Work in Progress) é um princípio essencial e uma prática central do Kanban. WIP são itens iniciados e não finalizados. O limite de WIP define a quantidade máxima de itens que podem estar simultaneamente em um mesmo estágio do fluxo. Essa prática é crucial porque reduz a multitarefa, evita a sobrecarga da equipe, ajuda a identificar gargalos e melhora o fluxo de trabalho. O limite de WIP não define a ordem das tarefas, apenas restringe a quantidade em andamento, favorecendo o foco e a eficiência. O WIP deve ser mantido continuamente para garantir estabilidade e previsibilidade no fluxo.

Gerenciamento do Fluxo

O Kanban não busca apenas manter um workflow padrão, mas sim melhorar continuamente o fluxo de trabalho. O gerenciamento do fluxo é uma prática essencial que envolve monitorar, medir e reportar o andamento das tarefas em cada estágio. Gargalos podem surgir em uma ou mais etapas do fluxo, e o gerenciamento contínuo é fundamental para identificá-los e otimizar a entrega de valor. O sistema puxado do Kanban faz com que novas tarefas sejam puxadas para o fluxo apenas quando há capacidade disponível, garantindo que novos itens só sejam iniciados quando houver capacidade, evitando sobrecarga e melhorando o fluxo de trabalho.

Políticas Explícitas

As políticas do Kanban podem e devem ser ajustadas conforme a evolução do trabalho. Isso significa que as regras para cada etapa do fluxo, como quando uma tarefa pode ser movida de uma coluna para outra, devem ser claras e explícitas para todos. Essa clareza ajuda a manter a consistência e a eficiência do processo.

Ciclos de Feedback e Melhorias

O Kanban incentiva fortemente revisões e melhorias contínuas. Recomenda cadências curtas de feedback para promover melhorias contínuas, não ciclos anuais. O feedback regular é importante para ajustar o processo com base em observações contínuas, garantindo que o fluxo seja sempre otimizado. O Kanban foca nas tarefas e favorece o aprimoramento contínuo dos processos.

Mudanças Evolutivas

O Kanban promove mudanças evolutivas, ajustando o processo com base em feedbacks contínuos para melhorar o fluxo e a entrega. Isso significa que as mudanças são graduais e incrementais, permitindo que a equipe se adapte sem grandes interrupções. O Kanban é flexível e adaptável, permitindo ajustes conforme as necessidades do projeto e do time.

Kanban x Scrum

INCIDÊNCIA EM PROVA: ALTA

Embora tanto Kanban quanto Scrum sejam métodos ágeis, eles possuem diferenças fundamentais em sua abordagem e práticas:

CARACTERÍSTICA	DESCRIÇÃO
ITERAÇÕES	O Scrum organiza o trabalho em iterações fixas chamadas sprints, com duração definida (geralmente de 1 a 4 semanas). O Kanban, por outro lado, opera com um fluxo contínuo, sem iterações temporais definidas ou prazos fixos para conclusão dos itens. A limitação de WIP não está ligada a sprints, mas a limitar itens em andamento em qualquer momento.
PAPÉIS	O Scrum possui papéis definidos e obrigatórios, como Product Owner, Scrum Master e Time de Desenvolvimento. No Kanban, não há papéis formais ou obrigatórios como esses, focando mais na visualização e no gerenciamento do fluxo.

EVENTOS	O Scrum prevê reuniões diárias obrigatórias (Daily Scrum), planejamento de sprint (Sprint Planning), revisão de sprint (Sprint Review) e retrospectiva de sprint (Sprint Retrospective). No Kanban, reuniões não são obrigatórias e são mais flexíveis, podendo ser adaptadas conforme a necessidade da equipe. O Kanban não elimina reuniões, apenas não as prescreve.
BACKLOG	No Scrum, o backlog pode ser ajustado entre sprints. No Kanban, o backlog pode ser ajustado de forma contínua, não apenas no fim do projeto.
LIMITES DE WIP	No Scrum, não há "limites automáticos de WIP". No Kanban, é possível (e recomendado) definir limites de WIP para otimizar o fluxo e evitar gargalos, diferindo do Scrum nesse aspecto.
INCREMENTOS	A definição de incrementos é prática do Scrum. No Kanban, não há incrementos fixos como releases obrigatórios; o trabalho flui de forma contínua.

Scrum & Kanban

INCIDÊNCIA EM PROVA: BAIXA

Kanban e Scrum podem ser usados juntos, formando o Scrumban. Essa abordagem combina elementos de ambos os métodos, como os sprints e papéis do Scrum com a visualização e os limites de WIP do Kanban. Não há incompatibilidade entre os dois métodos, e o Scrumban pode ser uma solução eficaz para equipes que buscam o melhor dos dois mundos.

Principais Métricas

INCIDÊNCIA EM PROVA: BAIXÍSSIMA

No Kanban, a medição do desempenho do fluxo de trabalho é fundamental para a melhoria contínua. Duas métricas principais são utilizadas para isso:

MÉTRICAS	DESCRIÇÃO
LEAD TIME	O lead time mede o tempo total que um item leva desde o início (quando é solicitado) até a entrega (sua conclusão). Essa métrica é crucial para avaliar a previsibilidade e a eficiência do processo, independentemente de iterações fixas como as sprints do Scrum. O lead time não é medido apenas a partir da coluna "Fazendo", mas sim desde o momento em que a tarefa é solicitada até sua finalização.
CYCLE TIME	O cycle time mede o tempo desde que um item entra em execução (início do trabalho ativo) até sua conclusão. Diferente do lead time, o cycle time foca no tempo que o item permanece nas etapas de "Fazendo" do quadro Kanban, aplicando-se apenas aos itens em andamento ou já finalizados. Essa métrica ajuda a entender a eficiência do processo de execução em si.

Kanban na Prática

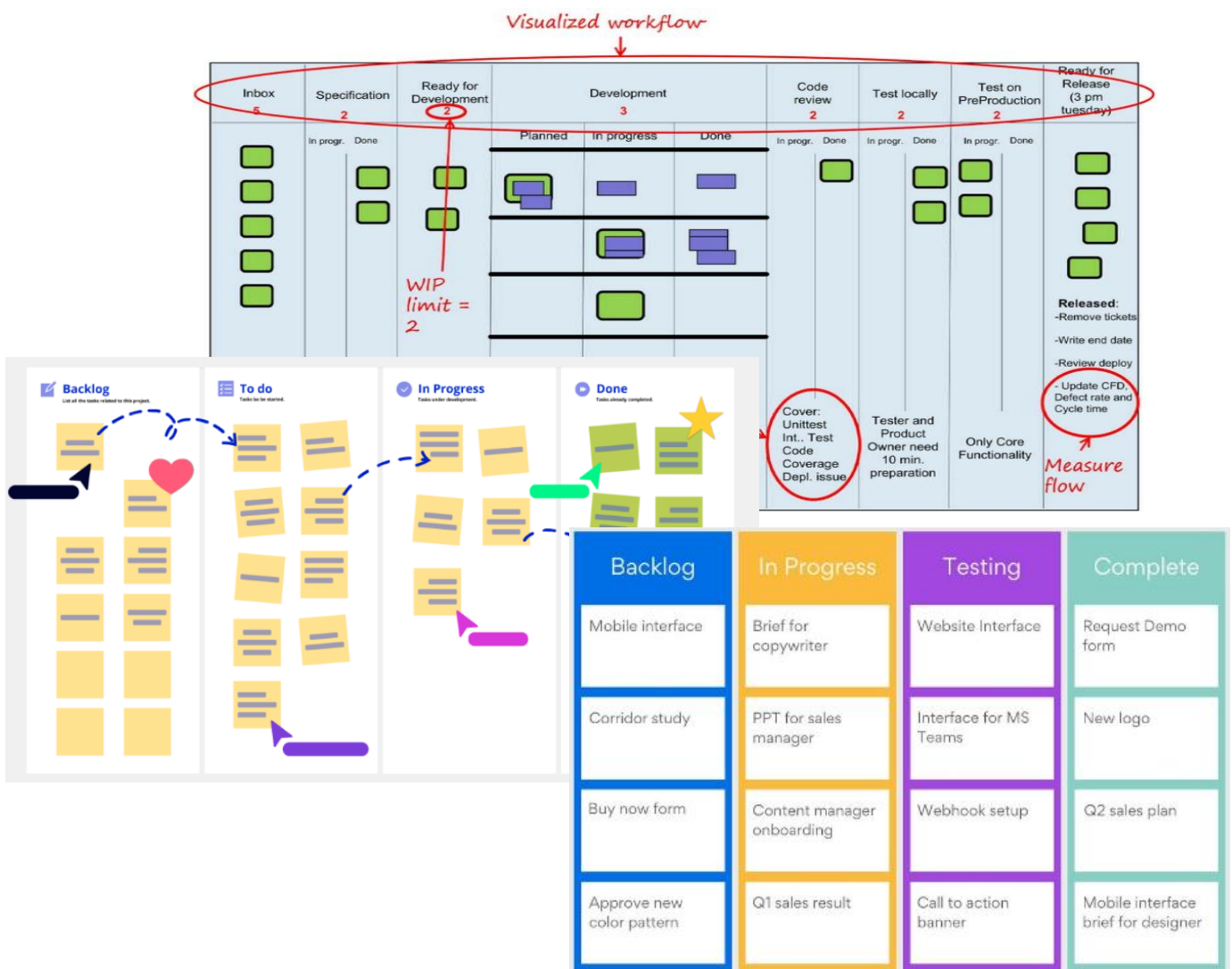
NCIDÊNCIA EM PROVA: MÉDIA

Para aplicar o Kanban de forma eficiente, é muito importante entender como suas práticas se manifestam no dia a dia:

Quadro Kanban

O quadro Kanban é a ferramenta visual central do método. Sua estrutura mais comum é composta por colunas como "A fazer", "Fazendo" e "Feito", representando o fluxo básico de trabalho. No entanto, a equipe define o fluxo de trabalho inicial com suas etapas, e esse fluxo pode ser ajustado ao longo do tempo, incluindo ou removendo estágios, conforme as necessidades e melhorias identificadas. O quadro mostra o fluxo de tarefas e ajuda a aumentar a visibilidade do fluxo por meio da representação visual.

É importante notar que o quadro não serve para controlar o esforço individual, mas para dar visibilidade ao fluxo de trabalho.



Gerenciamento de Gargalos

No Kanban, gargalos podem aparecer em qualquer etapa do processo. Por isso, o gerenciamento contínuo do fluxo é fundamental para identificar problemas e otimizar a entrega de valor. Limitar o WIP aumenta a visibilidade de gargalos e do andamento do trabalho, não a reduz. A visualização do fluxo no Kanban ajuda a identificar gargalos e obstáculos entre tarefas, permitindo melhorias. Limitar o WIP ajuda a identificar gargalos, melhorar a eficiência e reduzir tarefas acumuladas e inacabadas.

O Kanban adota conceitos da Teoria das Restrições, como a identificação e gestão de gargalos no fluxo de trabalho, para otimizar o processo e melhorar a entrega contínua de valor.

Reuniões no Kanban

Ao contrário do Scrum, o Kanban não prescreve reuniões diárias obrigatórias ou outras cerimônias fixas. No entanto, o Kanban não elimina reuniões; apenas não as prescreve. A equipe pode adaptar reuniões conforme a necessidade. Uma reunião comum no Kanban é o Internal Team Replenishment Meeting. Nesta reunião, a equipe escolhe quais itens de trabalho serão puxados para o fluxo a seguir, geralmente a partir do backlog ou quadro de opções disponíveis.

Ela acontece em uma cadência pré-definida (semanal ou conforme a demanda) e tem como objetivo garantir que o time tenha sempre trabalho priorizado pronto para iniciar, evitando paradas no fluxo. Durante o encontro, o grupo revisa os itens candidatos, verifica se estão suficientemente detalhados para execução, confirma prioridades e decide o que será adicionado à coluna de trabalho "Pronto para Iniciar" (Ready).

É um momento interno, focado no alinhamento da própria equipe, sem a necessidade de participação de stakeholders externos, e serve para manter um fluxo contínuo e previsível de entrega.

QUESTÕES COMENTADAS – FGV

1. (FGV / TJ-MT - 2024) No método Kanban, uma das práticas centrais ajuda a identificar gargalos e promover a eficiência ao limitar a quantidade de trabalho em progresso. O texto acima se refere à seguinte prática do Kanban:

- a) dividir o trabalho em iterações fixas.
- b) estabelecer metas rígidas de entrega.
- c) realizar reuniões diárias de acompanhamento.
- d) aumentar o número de colunas no quadro Kanban.
- e) limitar o WIP (Work in Progress).

Comentários:

- (a) Errado. Dividir o trabalho em iterações fixas é uma prática mais associada ao Scrum, que usa sprints, não ao Kanban, que é contínuo;
- (b) Errado. Metas rígidas de entrega não fazem parte do Kanban, que busca flexibilidade e fluxo contínuo de trabalho;
- (c) Errado. Reuniões diárias são comuns no Scrum (Daily Scrum), mas no Kanban não são obrigatórias;
- (d) Errado. Aumentar colunas no quadro não necessariamente melhora o fluxo ou reduz gargalos, podendo até aumentar a complexidade visual;
- (e) Correto. Limitar o WIP é prática central no Kanban, pois reduz multitarefa, ajuda a identificar gargalos e melhora o fluxo de trabalho.

Gabarito: Letra E

2. (FGV / TRF - 1ª REGIÃO - 2024) O Time de Soluções Inovadoras (TISI) de uma organização está utilizando práticas do Kanban no processo de desenvolvimento de soluções de software. Com o uso do Kanban, o TISI visa a:

- a) manter um workflow padrão para o processo de inovação;
- b) gerenciar e melhorar o desempenho dos indivíduos do time;
- c) estabelecer uma metodologia de desenvolvimento de soluções de software de modo iterativo;
- d) aplicar um processo para definição de incrementos para desenvolvimento de soluções de software;
- e) utilizar uma abordagem de mudança evolutiva, baseada em feedbacks, para o processo de desenvolvimento de soluções de software.

Comentários:

- (a) Errado. O Kanban não busca apenas manter um workflow padrão, mas sim melhorar continuamente o fluxo de trabalho;
- (b) Errado. O foco do Kanban é no fluxo do trabalho, não no desempenho individual dos membros do time;
- (c) Errado. Desenvolvimento iterativo é mais característico de metodologias como Scrum, não do Kanban;
- (d) Errado. A definição de incrementos é prática do Scrum, não um objetivo principal do Kanban;
- (e) Correto. O Kanban promove mudanças evolutivas, ajustando o processo com base em feedbacks contínuos para melhorar o fluxo e a entrega.

Gabarito: Letra E**3. (FGV / TCE-RR - 2025) No contexto da gestão de projetos com metodologias ágeis, como Scrum e Kanban, avalie as afirmativas a seguir.**

I. O Scrum é baseado em um fluxo contínuo de tarefas, em que as prioridades são ajustadas dinamicamente durante o projeto, enquanto o Kanban utiliza Sprints com duração fixa para organizar o trabalho em ciclos fechados.

II. O Scrum segue uma estrutura predefinida com cerimônias específicas, como Sprint Planning e Daily Standup, enquanto o Kanban é mais flexível, focando no controle do fluxo de trabalho por meio de limites no Work In Progress (WIP).

III. No Kanban, os papéis de Product Owner e Scrum Master são essenciais para o funcionamento do método, enquanto no Scrum essas funções são opcionais e podem ser combinadas em um único papel.

Está correto o que se afirma em

- a) I, apenas.
- b) II, apenas.
- c) III, apenas.
- d) I e II, apenas.
- e) II e III, apenas.

Comentários:

- (I) Errado. Essa descrição inverte os conceitos: o fluxo contínuo é característico do Kanban, e o uso de sprints fixos pertence ao Scrum;
- (II) Correto. O Scrum possui eventos e papéis definidos, enquanto o Kanban é mais flexível e utiliza limites de WIP para gerenciar o fluxo;
- (III) Errado. No Kanban não há papéis obrigatórios como Product Owner ou Scrum Master; esses são exclusivos do Scrum e são funções essenciais nele.

Gabarito: Letra B

4. (FGV / SEFAZ-MT - 2023) Muitas organizações aplicam os princípios e práticas de gestão com Kanban para alcançar os objetivos de seus projetos de desenvolvimento de software. Sobre Kanban, assinale a afirmativa correta.

- a) Incentiva ciclos anuais de feedbacks porque cadências de reuniões e revisões recorrentes criam muitas interrupções e impactam o tempo de entrega.
- b) É um método de gestão baseado no Kaizen e que valoriza entrega de software funcional completo, sem entregas parciais, trabalhos inacabados e débitos técnico.
- c) Fomenta uma cultura organizacional que privilegia o indivíduo em relação à coletividade, orientada ao cliente e sem compartilhar propósitos e metas.
- d) Pressupõe que o escopo do projeto é sempre estável e o desenvolvimento de software precisa seguir de modo linear para garantir a produtividade.
- e) Permite visualizar o processo e o fluxo de trabalho de projetos, programas e portfólios por meio de um conjunto de quadros interconectados.

Comentários:

- (a) Errado. O Kanban recomenda cadências curtas de feedback para promover melhorias contínuas, não ciclos anuais;
- (b) Errado. Embora inspirado no Kaizen, o Kanban permite entregas incrementais e contínuas, não exige apenas software completo;
- (c) Errado. O Kanban incentiva colaboração e alinhamento, não prioriza o indivíduo sobre a equipe;
- (d) Errado. O Kanban é flexível quanto ao escopo e não segue um modelo linear;

(e) Correto. Uma das práticas centrais do Kanban é a visualização do fluxo de trabalho, muitas vezes com quadros interconectados, para gerenciar e melhorar processos.

Gabarito: Letra E

5. (FGV / TJ-DFT - 2022) A Equipe de Gestão de Dados (EGD) de um órgão público optou por aplicar práticas ágeis em seus projetos. Uma das propostas da EGD é utilizar o sistema de gestão Kanban para observar de forma contínua o fluxo do trabalho, de modo a:

- a) fixar a duração de um ciclo de entrega;
- b) identificar possíveis obstáculos entre as tarefas;
- c) eliminar os Epics com User Stories complexas do Backlog;
- d) garantir que o quadro Kanban apresente os Backlogs dos projetos em andamento;
- e) permitir que os membros da equipe executem mais de uma tarefa por vez (WIP - work in progress).

Comentários:

- (a) Errado. Fixar duração de ciclos é prática do Scrum, não do Kanban, que trabalha com fluxo contínuo;
- (b) Correto. A visualização do fluxo no Kanban ajuda a identificar gargalos e obstáculos entre tarefas, permitindo melhorias;
- (c) Errado. Kanban não exige a exclusão de épicos ou histórias complexas do backlog;
- (d) Errado. O quadro Kanban mostra o fluxo de tarefas, não obrigatoriamente todos os backlogs dos projetos;
- (e) Errado. Limitar o WIP é prática central do Kanban, justamente para evitar que membros assumam muitas tarefas ao mesmo tempo.

Gabarito: Letra B

6. (FGV / TJ-RJ - 2024) Diversas empresas têm adotado o Kanban como ferramenta de aumento do fluxo e da produtividade no desenvolvimento de software. Considerando que esta tecnologia requer a adesão a algumas práticas fundamentais, avalie se as afirmativas a seguir são verdadeiras (V) ou falsas (F).

() Os quadros Kanban incorporam o princípio da visualização do trabalho que se baseia na exibição de cartões que correspondem a itens da lista de pendências do produto.

() Empregando o modelo de Pull a equipe puxa para seu fluxo de trabalho as pendências da lista conforme a sua capacidade se torna disponível.

() A imposição de limite para o número de tarefas que um time possui atualmente corresponde ao WIP (Work In Progress), e traz o benefício de aumentar o foco e, ao mesmo tempo, reduzir a mudança de contexto.

As afirmativas são, respectivamente,

- a) F - V - F.
- b) V - F - F.
- c) F - F - V.
- d) V - V - F.
- e) V - V - V.

Comentários:

(V) No Kanban, a visualização do trabalho é feita em um quadro com cartões representando itens do backlog ou tarefas;

(V) O sistema *pull* do Kanban faz com que novas tarefas sejam puxadas para o fluxo apenas quando há capacidade disponível;

(V) Limitar o WIP reduz multitarefa e mudanças de contexto, aumentando o foco e eficiência.

Gabarito: Letra E

7. (FGV / TRF - 1ª REGIÃO - 2024) O Departamento de Capacitação (DECAP) é responsável pela gestão do aprimoramento profissional dos colaboradores de uma determinada organização. O DECAP, que planeja, coordena e executa treinamentos, está aplicando Kanban como método de gerenciamento de seu processo de trabalho. O DECAP estabeleceu um Work in Progress Limit (WIP Limit) na gestão de seu processo de negócio visando a:

- a) associá-lo a práticas ágeis de entrega contínua;
- b) planejar e iniciar mais treinamentos em menos tempo;
- c) criar um equilíbrio entre demanda e capacidade ao longo do tempo;
- d) identificar os treinamentos que ultrapassam o limite estabelecido;
- e) definir o período máximo de tempo que uma etapa do processo deve durar.

Comentários:

- (a) Errado. Embora o Kanban possa se associar à entrega contínua, o WIP Limit não é definido com esse objetivo principal;
- (b) Errado. O limite de WIP visa evitar iniciar muitas atividades ao mesmo tempo, não aumentar o volume iniciado;
- (c) Correto. O WIP Limit equilibra a demanda de trabalho com a capacidade da equipe, evitando sobrecarga e mantendo fluxo estável;
- (d) Errado. Identificar treinamentos que ultrapassam o limite é consequência, não o objetivo principal do WIP Limit;
- (e) Errado. O WIP Limit não define tempo máximo por etapa – isso se relaciona a métricas como *lead time* ou *cycle time*.

Gabarito: Letra C

8. (FGV / MPE-RJ - 2025) A mensuração da métrica lead time no desenvolvimento de software ganhou notoriedade quando David Anderson, criador do Kanban destacou a importância de coletá-la. Com relação à utilidade de medir o lead time durante o processo de desenvolvimento de software, analise os itens a seguir

I. Analisar a saúde do processo de desenvolvimento considerando que altas dispersões representam algum tipo de gargalo ou aumento no tempo de passagem em alguma das etapas do fluxo de desenvolvimento, por exemplo, nas últimas semanas, o lead time das histórias de desenvolvimento cresceram, pois o ambiente de homologação estava com problemas e os testes eram mais complexos.

II. Identificar casos extremos (outliers) e aprender com o ocorrido, por exemplo, se um bug levou muito mais tempo do que o normal para ser corrigido em decorrência da ausência de clareza sobre o que era de fato o problema a ser resolvido.

III. Para compreender os efeitos que as incertezas e as complexidades não mapeadas podem causar, na forma de variabilidade, no tempo necessário para a conclusão dos trabalhos de um time de desenvolvimento de software.

Está correto o que se afirma em

- a) I, II e III.
- b) II e III, apenas.
- c) I e III, apenas.
- d) I e II, apenas.

e) III, apenas.

Comentários:

(I) Correto. O lead time ajuda a detectar gargalos e problemas em etapas específicas do fluxo;
(II) Correto. Casos extremos revelam pontos críticos que podem ser analisados para melhoria;
(III) Correto. A análise do lead time permite avaliar impactos de incertezas e complexidades na duração do trabalho.

Gabarito: Letra A

9. (FGV / Banestes - 2021) Observe o quadro comparativo a seguir, publicado em sites ligados ao estudo e à investigação de diferentes estratégias/metodologias para implementar um sistema ágil de desenvolvimento ou gestão de projetos. É correto identificar que X e Y representam, respectivamente:

Aspectos	X	Y
Ritmo	Sprints	Fluxo contínuo
Funções	Funções bem definidas	Sem funções necessárias
Entregas	Final de cada sprint	Entrega contínua
Mudanças	Evitar durante sprint	A qualquer momento

- a) Crystal e Scrum;
- b) Extreme Programming e Crystal;
- c) Kanban e Lean;
- d) Lean e Extreme Programming;
- e) Scrum e Kanban.

Comentários:

No quadro, X apresenta características como sprints, funções bem definidas, entregas no final de cada sprint e restrição de mudanças – características do **Scrum**.

Já Y mostra fluxo contínuo, ausência de funções obrigatórias, entrega contínua e possibilidade de mudanças a qualquer momento – características do **Kanban**.

Gabarito: Letra E

10.(FGV / SEFAZ-AM - 2022) Maria é líder de uma equipe de desenvolvimento de software. Para priorizar a produtividade e a organização das entregas, ela escolheu adotar um método de gestão visual para controle das tarefas e do fluxo de trabalho baseado na utilização de cartões que descrevem as atividades e um mural dividido em três seções rotuladas da seguinte forma: “para fazer”, “fazendo” e “feito”. Conforme o processo de desenvolvimento avança, os cartões são reposicionados no mural de modo a permitir que a equipe tenha uma dimensão do que está sendo produzido e em que ritmo está sendo produzido. No contexto da Engenharia de Software, é correto afirmar que Maria utilizou o método

- a) FDD.
- b) Gantt.
- c) Kanban.
- d) Fishbone.
- e) Seis sigma.

Comentários:

O cenário descreve claramente o uso de um quadro visual com cartões e colunas “para fazer”, “fazendo” e “feito”, característico do método Kanban, que organiza e controla o fluxo de trabalho de forma visual.

Gabarito: Letra C

LISTA DE QUESTÕES – FGV

1. (FGV / TJ-MT - 2024) No método Kanban, uma das práticas centrais ajuda a identificar gargalos e promover a eficiência ao limitar a quantidade de trabalho em progresso. O texto acima se refere à seguinte prática do Kanban:

- a) dividir o trabalho em iterações fixas.
- b) estabelecer metas rígidas de entrega.
- c) realizar reuniões diárias de acompanhamento.
- d) aumentar o número de colunas no quadro Kanban.
- e) limitar o WIP (Work in Progress).

2. (FGV / TRF - 1ª REGIÃO - 2024) O Time de Soluções Inovadoras (TISI) de uma organização está utilizando práticas do Kanban no processo de desenvolvimento de soluções de software. Com o uso do Kanban, o TISI visa a:

- a) manter um workflow padrão para o processo de inovação;
- b) gerenciar e melhorar o desempenho dos indivíduos do time;
- c) estabelecer uma metodologia de desenvolvimento de soluções de software de modo iterativo;
- d) aplicar um processo para definição de incrementos para desenvolvimento de soluções de software;
- e) utilizar uma abordagem de mudança evolutiva, baseada em feedbacks, para o processo de desenvolvimento de soluções de software.

3. (FGV / TCE-RR - 2025) No contexto da gestão de projetos com metodologias ágeis, como Scrum e Kanban, avalie as afirmativas a seguir.

I. O Scrum é baseado em um fluxo contínuo de tarefas, em que as prioridades são ajustadas dinamicamente durante o projeto, enquanto o Kanban utiliza Sprints com duração fixa para organizar o trabalho em ciclos fechados.

II. O Scrum segue uma estrutura predefinida com cerimônias específicas, como Sprint Planning e Daily Standup, enquanto o Kanban é mais flexível, focando no controle do fluxo de trabalho por meio de limites no Work In Progress (WIP).

III. No Kanban, os papéis de Product Owner e Scrum Master são essenciais para o funcionamento do método, enquanto no Scrum essas funções são opcionais e podem ser combinadas em um único papel.

Está correto o que se afirma em

- a) I, apenas.

- b) II, apenas.
- c) III, apenas.
- d) I e II, apenas.
- e) II e III, apenas.

4. (FGV / SEFAZ-MT - 2023) Muitas organizações aplicam os princípios e práticas de gestão com Kanban para alcançar os objetivos de seus projetos de desenvolvimento de software. Sobre Kanban, assinale a afirmativa correta.

- a) Incentiva ciclos anuais de feedbacks porque cadências de reuniões e revisões recorrentes criam muitas interrupções e impactam o tempo de entrega.
- b) É um método de gestão baseado no Kaizen e que valoriza entrega de software funcional completo, sem entregas parciais, trabalhos inacabados e débitos técnico.
- c) Fomenta uma cultura organizacional que privilegia o indivíduo em relação à coletividade, orientada ao cliente e sem compartilhar propósitos e metas.
- d) Pressupõe que o escopo do projeto é sempre estável e o desenvolvimento de software precisa seguir de modo linear para garantir a produtividade.
- e) Permite visualizar o processo e o fluxo de trabalho de projetos, programas e portfólios por meio de um conjunto de quadros interconectados.

5. (FGV / TJ-DFT - 2022) A Equipe de Gestão de Dados (EGD) de um órgão público optou por aplicar práticas ágeis em seus projetos. Uma das propostas da EGD é utilizar o sistema de gestão Kanban para observar de forma contínua o fluxo do trabalho, de modo a:

- a) fixar a duração de um ciclo de entrega;
- b) identificar possíveis obstáculos entre as tarefas;
- c) eliminar os Epics com User Stories complexas do Backlog;
- d) garantir que o quadro Kanban apresente os Backlogs dos projetos em andamento;
- e) permitir que os membros da equipe executem mais de uma tarefa por vez (WIP - work in progress).

6. (FGV / TJ-RJ - 2024) Diversas empresas têm adotado o Kanban como ferramenta de aumento do fluxo e da produtividade no desenvolvimento de software. Considerando que esta tecnologia requer a adesão a algumas práticas fundamentais, avalie se as afirmativas a seguir são verdadeiras (V) ou falsas (F).

() Os quadros Kanban incorporam o princípio da visualização do trabalho que se baseia na exibição de cartões que correspondem a itens da lista de pendências do produto.

() Empregando o modelo de Pull a equipe puxa para seu fluxo de trabalho as pendências da lista conforme a sua capacidade se torna disponível.

() A imposição de limite para o número de tarefas que um time possui atualmente corresponde ao WIP (Work In Progress), e traz o benefício de aumentar o foco e, ao mesmo tempo, reduzir a mudança de contexto.

As afirmativas são, respectivamente,

- a) F - V - F.
- b) V - F - F.
- c) F - F - V.
- d) V - V - F.
- e) V - V - V.

7. (FGV / TRF - 1ª REGIÃO - 2024) O Departamento de Capacitação (DECAP) é responsável pela gestão do aprimoramento profissional dos colaboradores de uma determinada organização. O DECAP, que planeja, coordena e executa treinamentos, está aplicando Kanban como método de gerenciamento de seu processo de trabalho. O DECAP estabeleceu um Work in Progress Limit (WIP Limit) na gestão de seu processo de negócio visando a:

- a) associá-lo a práticas ágeis de entrega contínua;
- b) planejar e iniciar mais treinamentos em menos tempo;
- c) criar um equilíbrio entre demanda e capacidade ao longo do tempo;
- d) identificar os treinamentos que ultrapassam o limite estabelecido;
- e) definir o período máximo de tempo que uma etapa do processo deve durar.

8. (FGV / MPE-RJ - 2025) A mensuração da métrica lead time no desenvolvimento de software ganhou notoriedade quando David Anderson, criador do Kanban destacou a importância de coletá-la. Com relação à utilidade de medir o lead time durante o processo de desenvolvimento de software, analise os itens a seguir

I. Analisar a saúde do processo de desenvolvimento considerando que altas dispersões representam algum tipo de gargalo ou aumento no tempo de passagem em alguma das etapas do fluxo de desenvolvimento, por exemplo, nas últimas semanas, o lead time das histórias de desenvolvimento cresceram, pois o ambiente de homologação estava com problemas e os testes eram mais complexos.

II. Identificar casos extremos (outliers) e aprender com o ocorrido, por exemplo, se um bug levou muito mais tempo do que o normal para ser corrigido em decorrência da ausência de clareza sobre o que era de fato o problema a ser resolvido.

III. Para compreender os efeitos que as incertezas e as complexidades não mapeadas podem causar, na forma de variabilidade, no tempo necessário para a conclusão dos trabalhos de um time de desenvolvimento de software.

Está correto o que se afirma em

- a) I, II e III.
- b) II e III, apenas.
- c) I e III, apenas.
- d) I e II, apenas.
- e) III, apenas.

9. (FGV / Banestes - 2021) Observe o quadro comparativo a seguir, publicado em sites ligados ao estudo e à investigação de diferentes estratégias/metodologias para implementar um sistema ágil de desenvolvimento ou gestão de projetos. É correto identificar que X e Y representam, respectivamente:

Aspectos	X	Y
Ritmo	Sprints	Fluxo contínuo
Funções	Funções bem definidas	Sem funções necessárias
Entregas	Final de cada sprint	Entrega contínua
Mudanças	Evitar durante sprint	A qualquer momento

- a) Crystal e Scrum;
- b) Extreme Programming e Crystal;
- c) Kanban e Lean;
- d) Lean e Extreme Programming;
- e) Scrum e Kanban.

10.(FGV / SEFAZ-AM - 2022) Maria é líder de uma equipe de desenvolvimento de software. Para priorizar a produtividade e a organização das entregas, ela escolheu adotar um método de gestão visual para controle das tarefas e do fluxo de trabalho baseado na utilização de cartões que descrevem as atividades e um mural dividido em três seções rotuladas da seguinte forma: "para fazer", "fazendo" e "feito". Conforme o processo de desenvolvimento avança, os cartões são reposicionados no mural de modo a permitir que a equipe tenha uma dimensão do que está sendo produzido e em que ritmo está sendo produzido. No contexto da Engenharia de Software, é correto afirmar que Maria utilizou o método

- a) FDD.

- b) Gantt.
- c) Kanban.
- d) Fishbone.
- e) Seis sigma.

GABARITO – FGV

1. LETRA E
2. LETRA E
3. LETRA B
4. LETRA E
5. LETRA B
6. LETRA E
7. LETRA C
8. LETRA A
9. LETRA E
10. LETRA C

TEST DRIVEN DEVELOPMENT (TDD)

Conceitos Básicos

INCIDÊNCIA EM PROVA: MÉDIA

O Test-Driven Development (TDD) é uma abordagem de desenvolvimento de software em que **se intercalam testes e desenvolvimento de código**. Essencialmente, você desenvolve um código de forma incremental, em conjunto com um teste para esse incremento. Você não caminha para o próximo incremento até que o código desenvolvido passe no teste. O desenvolvimento dirigido a testes foi apresentado como parte dos métodos ágeis, como o XP.

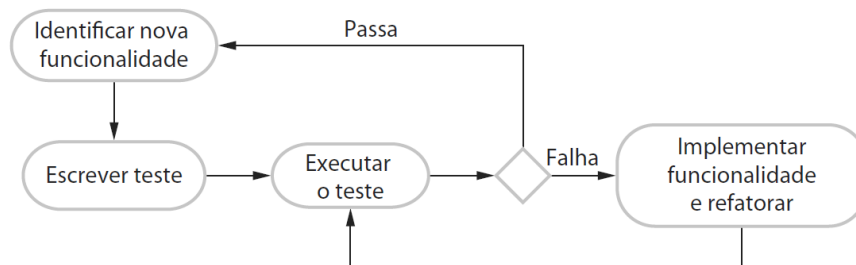
Por outro lado, ele também pode ser utilizado em processos de desenvolvimento dirigido a planos. Trata-se de uma abordagem que **se baseia na repetição de um ciclo de desenvolvimento curto focado em testes unitários**. A ideia fundamental dessa abordagem consiste em escrever o teste, encontrar uma falha nesse mesmo teste e depois refatorá-lo (caso necessário – não é obrigatório). Vamos agora ver as etapas do processo de desenvolvimento dirigido a testes:

ETAPA	DESCRIÇÃO
1	Você começa identificando o incremento de funcionalidade necessário. Este, normalmente, deve ser pequeno e implementável em poucas linhas de código.
2	Você escreve um teste para essa funcionalidade e o implementa como um teste automatizado. Isso significa que o teste pode ser executado e relatará se passou ou falhou.
3	Você, então, executa o teste, junto com todos os outros testes implementados. Inicialmente, você não terá implementado a funcionalidade, logo o novo teste falhará. Isso é proposital, pois mostra que o teste acrescenta algo ao conjunto de testes.
4	Você, então, implementa a funcionalidade e executa novamente o teste. Isso pode envolver a refatoração do código existente para melhorá-lo e adicionar um novo código sobre o que já está lá.
5	Depois que todos os testes forem executados com sucesso, você caminha para implementar a próxima parte da funcionalidade.

Como o código é desenvolvido em incrementos muito pequenos, você precisa ser capaz de executar todos os testes cada vez que adicionar funcionalidade ou refatorar o programa. Dessa forma, os testes são embutidos em um programa separado que os executa e invoca o sistema que está sendo testado. Usando essa abordagem, é possível rodar centenas e centenas de testes separados em poucos segundos.

Um argumento forte a favor do desenvolvimento dirigido a testes é que ele ajuda os programadores a clarear suas ideias sobre o que um segmento de código supostamente deve fazer. **Para escrever um teste, você precisa entender a que ele se destina, e como esse entendimento faz que seja**

mais fácil escrever o código necessário. Certamente, se você tem conhecimento ou compreensão incompleta, o desenvolvimento dirigido a testes não ajudará.



Se você não sabe o suficiente para escrever os testes, não vai desenvolver o código necessário. Por exemplo: se seu cálculo envolve divisão, você deve verificar se não está dividindo o número por zero. Se você se esquecer de escrever um teste para isso, então o código para essa verificação nunca será incluído no programa. Além de um melhor entendimento do problema, outros benefícios do desenvolvimento dirigido a testes são:

BENEFÍCIOS	DESCRIÇÃO
COBERTURA DE CÓDIGO	Em princípio, todo segmento de código que você escreve deve ter pelo menos um teste associado. Assim, você pode ter certeza de que todo o código no sistema foi realmente executado. Cada código é testado enquanto está sendo escrito; assim, os defeitos são descobertos no início do processo de desenvolvimento.
TESTE DE REGRESSÃO	Um conjunto de testes é desenvolvido de forma incremental enquanto um programa é desenvolvido. Você sempre pode executar testes de regressão para verificar se as mudanças no programa não introduziram novos bugs.
DEPURAÇÃO SIMPLIFICADA	Quando um teste falha, a localização do problema deve ser óbvia. O código recém-escrito precisa ser verificado e modificado. Você não precisa usar as ferramentas de depuração para localizar o problema. Alguns relatos de uso de desenvolvimento dirigido a testes sugerem que, em desenvolvimento dirigido a testes, quase nunca é necessário usar um sistema automatizado de depuração.
DOCUMENTAÇÃO DE SISTEMA	Os testes em si mesmos agem como uma forma de documentação que descreve o que o código deve estar fazendo. Ler os testes pode tornar mais fácil a compreensão do código.

Um dos benefícios mais importantes de desenvolvimento dirigido a testes é que ele reduz os custos dos testes de regressão. O teste de regressão envolve a execução de conjuntos de testes que tenham sido executados com sucesso, após as alterações serem feitas em um sistema. Ele também verifica se essas mudanças não introduziram novos bugs no sistema e se o novo código interage com o código existente conforme o esperado.

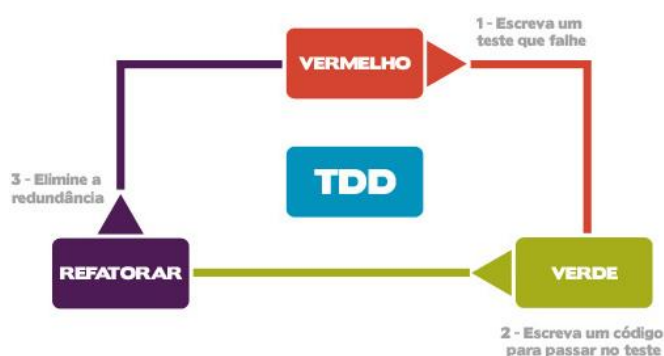
O teste de regressão é muito caro e geralmente impraticável quando um sistema é testado manualmente, pois os custos com tempo e esforço são muito altos. Em tais situações, você precisa tentar escolher os testes mais relevantes para executar novamente, e é fácil perder testes importantes. No entanto, testes automatizados – fundamentais para o desenvolvimento *test-first* – reduzem drasticamente os custos com testes de regressão.

Os testes existentes podem ser executados novamente de forma rápida e barata. **Após se fazer uma mudança para um sistema em desenvolvimento *test-first*, todos os testes existentes devem ser executados com êxito antes de qualquer funcionalidade ser adicionada.** Como um programador, você precisa ter certeza de que a nova funcionalidade não tenha causado ou revelado problemas com o código existente.

O desenvolvimento dirigido a testes é de maior utilidade no desenvolvimento de softwares novos, em que a funcionalidade seja implementada no novo código ou usando bibliotecas-padrão já testadas. **Se você estiver reusando componentes de código ou sistemas legados grandes, você precisa escrever testes para esses sistemas como um todo.** O desenvolvimento dirigido a testes também pode ser ineficaz em sistemas multi-threaded.

As *threads* diferentes podem ser intercalados em tempos diferentes, em execuções diferentes, e isso pode produzir resultados diferentes. Se você usa o desenvolvimento dirigido a testes, ainda precisa de um processo de teste de sistema para validar o sistema, isto é, verificar se atende aos requisitos dos stakeholders. O teste de sistema também testa o desempenho, a confiabilidade, e verifica se o sistema não faz coisas que não deveria, como produzir resultados indesejados etc.

O desenvolvimento dirigido a testes revelou-se uma abordagem de sucesso para projetos de pequenas e médias empresas. Geralmente, os programadores que adotaram essa abordagem estão satisfeitos com ela e acham que é uma maneira mais produtiva de desenvolver softwares. Em alguns experimentos, foi mostrado que essa abordagem gera melhorias na qualidade do código. Bem, agora vamos falar sobre o ciclo vermelho, verde e refatoração.



ETAPA	DESCRIÇÃO
VERMELHO (RED)	Escreva um teste que falha: casos de teste são escritos sem que exista o recurso a ser testado, levando o teste a falhar;

VERDE (GREEN)

Escreva código para passar no teste: o recurso é implementado com o mínimo de código necessário para que passe no teste.

REFATORAR (REFACTOR)

Elimine redundâncias: o código que implementa o recurso é refinado e melhorado, sem que seja adicionada novas funcionalidades.

Vamos lá! Nós sabemos que – para cada parte da aplicação – adiciona-se um teste escrito antes mesmo do desenvolvimento do código em si. *Por que?* **Porque eles podem ajudar a reduzir riscos de possíveis problemas no código.** Executamos o teste e ele... falha! Ele deve necessariamente falhar! *Por que?* Ora, porque ele é o primeiro teste e você nem criou a funcionalidade ainda, logo ele não irá funcionar!

Então nós adicionamos uma nova funcionalidade ao sistema apenas para que ele passe no teste e execute novamente (agora ele deve passar no teste). **Então, nós adicionamos um novo teste e rodamos o teste anterior e esse novo teste.** Se algum deles falhar, modifica-se o código da funcionalidade e rodam-se todos os testes novamente, e assim por diante – nós já vimos aquela imagem anterior que mostra como tudo isso funciona...

Galera, vocês percebem que o feedback sobre a nova funcionalidade ocorre de maneira bem rápido? **Além disso, cria-se um código mais limpo, visto que o código para passar nos testes deve ser bastante simples.** Há mais segurança na correção de eventuais bugs; aumenta-se a produtividade, visto que se perde menos tempo com depuradores; e o código se torna mais flexível, menos acoplado e mais coeso.

Nós podemos afirmar que, em geral, utilizam-se testes unitários, testes de integração ou testes de aceitação – sendo os dois primeiros os mais comuns. **Algumas ferramentas que podem ser utilizadas para implementar o processo de desenvolvimento orientado a testes:** JUnit, TesteNG, PHPUnit, SimpleTest, NUnit, Jasmine, CUnit, PyUnit, etc. Pessoal, agora um detalhe que nós passamos direto sobre a abordagem de desenvolvimento...



O TEST DRIVEN DEVELOPMENT (TDD) NÃO É UMA ABORDAGEM PARA REALIZAR TESTES – TRATA-SE DE UMA ABORDAGEM PARA DESENVOLVER SOFTWARES. ELA PODE EVENTUALMENTE SER CONSIDERADA TAMBÉM UMA TÉCNICA DE PROGRAMAÇÃO!

Professor, uma curiosidade: isso já caiu em alguma prova discursiva? **Sim, galera... o enunciado dessa prova requisitava ao aluno informar as vantagens do emprego do TDD em relação a outras metodologias ágeis.** Poderíamos responder essa pergunta afirmando que o software

desenvolvido, em geral, apresenta maior qualidade, na medida em que é implementado direcionado às expectativas do cliente.

Poderíamos dizer também que há a possibilidade de se testar todo o código desenvolvido, o que oferece maior confiabilidade ao sistema. Por fim, em geral, o código é mais modularizado, flexível e extensível, visto que a metodologia requer que os desenvolvedores imaginem o software como pequenas unidades¹ que podem ser reescritas, desenvolvidas e testadas de forma independente e integradas em momento posterior.

Essa mesma prova perguntava também quais são os princípios da Metodologia Extreme Programming (XP) apoiados pelo TDD. Uma resposta adequada poderia afirmar que o XP apresenta diversas práticas que podem ser relacionadas com o TDD. **Qual é a mais óbvia? A mais óbvia é o Test-First (Teste Primeiro), ratificando a característica básica recomendada veementemente pelo desenvolvimento orientado a testes.**

O TDD pode apoiar esse princípio por fornecer detalhes para a realização dos testes de unidade e de funcionalidade, que são importantes e necessários. Ademais, o desenvolvimento orientado a testes apresenta relação intrínseca com a refatoração, tendo em vista que confere ao programador maior segurança para identificar e remover o código duplicado, e permite, assim, a melhoria contínua do programa. Vamos resumir as principais características do TDD:

CARACTERÍSTICAS DO TDD	DESCRIÇÃO
ORIENTADO A TESTES	O TDD coloca o teste no centro do processo de desenvolvimento, guiando a escrita do código. Cada nova funcionalidade é conduzida por um ou mais testes que definem o comportamento esperado.
PEQUENOS PASSOS	TDD promove a implementação em pequenos passos iterativos. Cada ciclo de desenvolvimento é curto, envolvendo a escrita de um pequeno teste, o código correspondente e a refatoração.
ALTA COBERTURA	Como os testes são escritos antes do código, TDD naturalmente resulta em uma alta cobertura de testes automatizados, o que ajuda a detectar regressões e garantir que o código funcione conforme esperado.
FEEDBACK IMEDIATO	O TDD oferece feedback imediato sobre a correção do código. Se um teste falha, o desenvolvedor pode corrigir o problema imediatamente, minimizando o tempo entre a introdução de um erro e sua correção.
DOCUMENTAÇÃO VIVA	Os testes servem como documentação executável do comportamento do sistema. Eles descrevem o que o código deve fazer de forma precisa e concisa, e essa "documentação" é sempre atualizada porque os testes são executados continuamente.
FACILITA REFATORAÇÃO SEGURA	A prática de TDD promove um design de código mais limpo e modular, já que o desenvolvedor refatora o código regularmente com a segurança de que os testes irão capturar qualquer erro introduzido.

¹ Atenção: apesar de serem tipicamente realizados testes de unidade, não é obrigatória a utilização desse tipo de teste.

PROJETO GUIADO POR TESTES	TDD influencia positivamente o design do software, encorajando a criação de código coeso e de baixo acoplamento. Escrever testes primeiro ajuda os desenvolvedores a pensar na interface e na responsabilidade das classes antes de implementá-las.
MAIOR QUALIDADE DE CÓDIGO	A prática constante de escrever testes e refatorar resulta em um código de maior qualidade, com menos bugs e mais fácil de manter e evoluir.
REDUÇÃO DE BUGS DE PRODUÇÃO	Com o TDD, muitos erros são capturados e corrigidos durante o desenvolvimento, o que resulta em uma menor probabilidade de bugs aparecerem em produção.
MELHORIA NA CONFIANÇA	Como os testes automatizados garantem que o código funciona conforme esperado, os desenvolvedores têm mais confiança em modificar, refatorar ou estender o código sem introduzir novos erros.
AUMENTO DA MANUTENIBILIDADE	O código desenvolvido com TDD tende a ser mais modular e de fácil manutenção, uma vez que os testes forçam o desenvolvedor a pensar em como dividir o código em partes bem definidas e testáveis.
ASSERÇÕES DE VERIFICAÇÃO	Cada caso de teste deve incluir asserções que verifiquem se o comportamento do código está conforme o esperado. As asserções são fundamentais para garantir que o teste valide corretamente o comportamento do código.
TESTES DE INTEGRAÇÃO	Testes de integração são utilizados para validar recursos complexos, como acesso a dados, onde múltiplos componentes ou sistemas interagem. Esses testes garantem que as partes integradas funcionem corretamente juntas.
INDEPENDÊNCIA DOS TESTES	A independência dos testes é um princípio fundamental. Cada teste deve ser executado isoladamente, sem dependências entre eles. Comentários sobre quais testes foram criados não substituem a necessidade de garantir que os testes não se afetam mutuamente.

Galera, nós temos também uma variação chamada Acceptance Test-Driven Development (ATDD). **Ele é método ágil de desenvolvimento de software que se baseia na comunicação entre clientes do negócio, desenvolvedores e testadores.** Diferente do Test-Driven Development (TDD), que se concentra em testes unitários, o ATDD se concentra em testes de aceitação que validam se o sistema atende aos critérios de aceitação definidos pelo cliente ou pelo usuário final.

Ele promove uma estreita colaboração entre todas as partes interessadas (stakeholders). Isso inclui desenvolvedores, testadores, analistas de negócios e clientes que trabalham juntos para definir e escrever os testes de aceitação antes que o desenvolvimento comece. Essa colaboração garante que todos tenham um entendimento comum do que precisa ser construído. Os testes de aceitação servem como uma forma de documentação viva e executável.

Eles descrevem o comportamento esperado do sistema e podem ser usados para verificar automaticamente se o software atende aos requisitos acordados. Além disso, são diretamente derivados dos requisitos funcionais e não funcionais do sistema. Isso garante que o software entregue esteja em conformidade com as especificações e expectativas do cliente. Outra característica importante é o seu feedback imediato.

Como os testes são escritos antes do desenvolvimento, o ATDD oferece feedback imediato sobre a conformidade do software com os requisitos do cliente durante o processo de desenvolvimento.

Isso permite que problemas e mal-entendidos sejam identificados e corrigidos mais cedo. Ao validar os critérios de aceitação antes de começar a codificar, o ATDD ajuda a reduzir a quantidade de retrabalho causado por mal-entendidos sobre os requisitos.

Embora similar ao TDD, o ciclo do ATDD adapta o ciclo Red-Green-Refactor para se concentrar em testes de aceitação. O ciclo pode ser descrito como:

ETAPA	DESCRIÇÃO
RED	Escrever um teste de aceitação que inicialmente falha, porque a funcionalidade ainda não foi implementada.
GREEN	Implementar o código necessário para fazer o teste passar.
REFACTOR	Melhorar o código enquanto se certifica de que todos os testes de aceitação continuam a passar.

O uso do ATDD resulta em um software de maior qualidade, pois os requisitos do cliente são continuamente validados ao longo do processo de desenvolvimento. A confiança na qualidade do software também é aumentada, pois os testes de aceitação garantem que o sistema se comporta conforme o esperado. A tabela exibida a seguir apresenta as principais características do ATDD e a tabela seguinte nos mostra as diferenças para o TDD:

CARACTERÍSTICAS DO ATDD	DESCRIÇÃO
FOCO EM TESTES DE ACEITAÇÃO	ATDD envolve a escrita de testes de aceitação antes do desenvolvimento, garantindo que os critérios do cliente sejam atendidos.
COLABORAÇÃO ENTRE EQUIPES	Promove a colaboração entre desenvolvedores, testadores e stakeholders para definir os critérios de aceitação.
TESTES BASEADOS EM REQUISITOS	Os testes de aceitação são diretamente derivados dos requisitos funcionais, garantindo que o software entregue esteja alinhado com as expectativas do cliente.
DOCUMENTAÇÃO EXECUTÁVEL	Os testes de aceitação servem como documentação viva e executável, descrevendo o comportamento esperado do sistema.
FEEDBACK IMEDIATO	Oferece feedback imediato sobre a conformidade do software com os requisitos do cliente durante o desenvolvimento.
CICLO ADAPTADO	Similar ao TDD, mas com foco em garantir que o software atenda aos critérios de aceitação antes de refatorar o código.
ENGAJAMENTO DE STAKEHOLDERS	Incentiva o envolvimento dos stakeholders no processo de desenvolvimento, garantindo que o produto final atenda às suas necessidades.
QUALIDADE DO PRODUTO	Ao focar nos testes de aceitação desde o início, ATDD ajuda a detectar problemas cedo, resultando em um produto de maior qualidade.
REDUÇÃO DE RETRABALHO	Ao validar os critérios de aceitação antes do desenvolvimento, minimiza a necessidade de retrabalho causado por mal-entendidos nos requisitos.

VALIDAÇÃO DE REQUISITOS

Ajuda a validar e clarificar os requisitos antes de iniciar o desenvolvimento, garantindo que todos tenham o mesmo entendimento sobre as expectativas.

TDD (TEST-DRIVEN DEVELOPMENT)	ATDD (ACCEPTANCE TEST-DRIVEN DEVELOPMENT)
O foco principal do TDD está nos testes unitários. O desenvolvedor escreve pequenos testes automatizados antes de escrever o código correspondente. O objetivo é garantir que cada unidade de código (geralmente uma função ou método) funcione corretamente em um nível granular.	O ATDD se concentra nos testes de aceitação, que validam o comportamento do sistema em um nível mais alto, assegurando que o software atende aos requisitos e critérios de aceitação definidos pelos stakeholders, incluindo clientes, analistas de negócios e desenvolvedores.
Os testes em TDD são geralmente limitados a uma unidade de código individual, como um método ou classe. Eles são escritos pelos desenvolvedores para verificar a correção funcional de pequenas partes do sistema.	Os testes de ATDD abrangem o sistema como um todo ou grandes partes dele, focando no comportamento e na interação entre os componentes do sistema. Esses testes validam se a aplicação atende às necessidades e expectativas do usuário final.
A prática de TDD é geralmente realizada exclusivamente pelos desenvolvedores. O ciclo de escrita de testes, codificação e refatoração é iterado pelos programadores durante o desenvolvimento.	ATDD envolve uma colaboração mais ampla entre desenvolvedores, testadores, analistas de negócios e clientes. Todos trabalham juntos para definir os critérios de aceitação e os testes que garantem que o sistema está alinhado com as expectativas do cliente.
O objetivo do TDD é criar um código que funcione corretamente desde o nível mais baixo, permitindo um design de software mais limpo, modular e testável. Ele também visa capturar erros no nível mais granular possível.	O objetivo do ATDD é garantir que o sistema como um todo funcione conforme esperado pelos usuários finais e que ele atenda aos requisitos de negócios. ATDD valida que o software satisfaz as necessidades e critérios de aceitação antes de ser considerado "pronto".
Os testes criados durante o TDD atuam como uma forma de documentação técnica do comportamento das unidades de código. Eles ajudam outros desenvolvedores a entender como o código foi projetado para funcionar.	Os testes de aceitação no ATDD servem como documentação executável do sistema. Eles descrevem o comportamento esperado do sistema de uma maneira que é compreensível tanto para técnicos quanto para stakeholders não técnicos.
O ciclo de TDD (Red-Green-Refactor) é rápido e focado, proporcionando feedback imediato sobre pequenas porções de código. Isso ajuda a manter o desenvolvimento alinhado com o design desejado desde o início.	O ATDD também oferece feedback, mas em um nível mais alto, garantindo que o software em desenvolvimento está de acordo com os requisitos do usuário. O feedback no ATDD é obtido antes mesmo do início do desenvolvimento real, durante a fase de especificação dos requisitos.
Os testes são escritos pelos desenvolvedores, que também são responsáveis por escrever o código para passar esses testes.	Os testes são definidos em colaboração entre desenvolvedores, testadores e stakeholders (como clientes ou analistas de negócios), e podem ser escritos por desenvolvedores ou testadores.

Em suma: TDD é focado em garantir que as pequenas partes do código (unidades) funcionem corretamente, através da escrita de testes unitários antes do desenvolvimento do código correspondente. Ele é mais técnico e orientado para a implementação. ATDD é focado em garantir que o software como um todo atende aos requisitos do cliente, através da escrita de testes de aceitação antes do desenvolvimento. Ele é mais colaborativo e orientado para o negócio.

QUESTÕES COMENTADAS – DIVERSAS BANCAS

1. (CESPE / INMETRO – 2009) A rotina diária dos desenvolvedores, ao empregar processos baseados no TDD (Test-Driven Development), é concentrada na elaboração de testes de homologação.

Comentários:

A rotina dos desenvolvedores é concentrada, na verdade, na elaboração de testes unitários e, não, de homologação. Lembrem-se: constrói o teste, constrói a funcionalidade e refatora a funcionalidade.

Gabarito: Errado

2. (CESPE / INPI – 2013) Usando-se o TDD, as funcionalidades devem estar completas e da forma como serão apresentadas aos seus usuários para que possam ser testadas e consideradas corretas.

Comentários:

ETAPA	DESCRIÇÃO
VERMELHO (RED)	Escreva um teste que falha: casos de teste são escritos sem que exista o recurso a ser testado, levando o teste a falhar;
VERDE (GREEN)	Escreva código para passar no teste: o recurso é implementado com o mínimo de código necessário para que passe no teste.
REFATORAR (REFACTOR)	Elimine redundâncias: o código que implementa o recurso é refinado e melhorado, sem que seja adicionada novas funcionalidades.

Pelo contrário, primeiro são feitos os testes e depois desenvolvem-se as funcionalidades.

Gabarito: Errado

3. (CESPE / ANCINE – 2013) No desenvolvimento de software conforme as diretrizes do TDD (Test-Driven Development), deve-se elaborar primeiramente os testes e, em seguida, escrever o código necessário para passar pelos testes.

Comentários:

ETAPA	DESCRIÇÃO
VERMELHO (RED)	Escreva um teste que falha: casos de teste são escritos sem que exista o recurso a ser testado, levando o teste a falhar;

VERDE (GREEN)

Escreva código para passar no teste: o recurso é implementado com o mínimo de código necessário para que passe no teste.

REFATORAR (REFACTOR)

Elimine redundâncias: o código que implementa o recurso é refinado e melhorado, sem que seja adicionada novas funcionalidades.

Perfeito! Primeiro, criam-se os testes, depois cria-se o código.

Gabarito: Correto

4. (CESPE / INMETRO – 2009) Considerando uma organização na qual a abordagem de Test Driven Development (TDD) esteja implementada, assinale a opção correta.

- a) Nessa organização, ocorre a execução de iterações com ciclo longo, isto é, com duração de alguns meses.
- b) No início de cada iteração, a primeira atividade realizada pela equipe de desenvolvimento é produzir o código que será validado através de testes.
- c) O refactoring é uma das primeiras atividades realizada no início de cada iteração.
- d) Entre as atividades finais de cada iteração, o desenvolvedor escreve casos de teste automatizados, cuja execução verifica se houve a melhoria desejada ou se uma nova funcionalidade foi implementada.
- e) Há coerência e inter-relação com os princípios promovidos pela prática da extreme programming (XP).

Comentários:

(a) Errado, são ciclos curtos e, não, longos; (b) Errado, são produzidos primeiramente os testes e, depois, os códigos; (c) Errado, a refatoração é a última atividade realizada em uma iteração; (d) Errado, escrever casos de testes é uma das atividades iniciais; (e) Correto, trata-se inclusive de uma das práticas recomendadas pelo XP.

Gabarito: Letra E

5. (CESPE / MPOG – 2013) Ao realizar o TDD (test-driven development), o programador é conduzido a pensar em decisões de design antes de pensar em código de implementação, o que cria um maior acoplamento, uma vez que seu objetivo é pensar na lógica e nas responsabilidades de cada classe.

Comentários:

Em Engenharia de Software, há dois conceitos importantíssimos: coesão e acoplamento. Quando eu estudava, eu decorava uma frase pequena para entender: "*Coesão é a divisão de responsabilidades e Acoplamento é a dependência entre componentes*". Há outra que dizia assim: "*Uma boa arquitetura de software deve ter componentes de projeto com baixo acoplamento e alta coesão*".

O Acoplamento trata do nível de dependência entre módulos ou componentes de um software. *Por que é bom ter baixo acoplamento?* Porque se os módulos pouco dependem um do outro, modificações de um não afetam os outros, além de não prejudicar o reúso. Se esse princípio não for observado durante a construção da arquitetura de um sistema de software, pode haver problemas sérios de manutenção futura!

Voltando à questão: se o programador pensa em decisões de design antes de pensar em código de implementação, isso diminui o acoplamento - os componentes ficam menos dependentes, tornando a arquitetura bem mais flexível.

Gabarito: Errado

6. (CESPE / MPU – 2013) Na metodologia TDD, ou desenvolvimento orientado a testes, cada nova funcionalidade inicia com a criação de um teste, cujo planejamento permite a identificação dos itens e funcionalidades que deverão ser testados, quem são os responsáveis e quais os riscos envolvidos.

Comentários:

Perfeito! Essa metodologia permite um aprendizado maior sobre o problema a ser resolvido, permitindo (não obrigatoriamente) a identificação de itens, funcionalidades, responsáveis e riscos.

Gabarito: Correto

7. (CESPE / STF – 2013) No TDD, o primeiro passo do desenvolvedor é criar o teste, denominado teste falho, que retornará um erro, para, posteriormente, desenvolver o código e aprimorar a codificação do sistema.

Comentários:

ETAPA	DESCRIÇÃO
VERMELHO (RED)	Escreva um teste que falha: casos de teste são escritos sem que exista o recurso a ser testado, levando o teste a falhar;
VERDE (GREEN)	Escreva código para passar no teste: o recurso é implementado com o mínimo de código necessário para que passe no teste.
REFATORAR (REFACTOR)	Elimine redundâncias: o código que implementa o recurso é refinado e melhorado, sem que seja adicionada novas funcionalidades.

Perfeito! Cria-se o teste falho, desenvolve-se um código e só então ocorre a refatoração.

Gabarito: Correto

8. (CESPE / TRT17 – 2013) TDD consiste em uma técnica de desenvolvimento de software com abordagem embasada em perspectiva evolutiva de seu desenvolvimento. Essa abordagem envolve a produção de versões iniciais de um sistema a partir das quais é possível realizar verificações de suas qualidades antes que ele seja construído.

Comentários:

A questão diz que versões iniciais são produzidas e, a partir dessas versões, é possível realizar verificações de suas qualidades antes que ele seja construído. Na verdade, testes são criados inicialmente para verificar sua qualidade e, a partir daí, versões são produzidas. Logo, a questão inverteu os conceitos!

Gabarito: Correto

9. (CESPE / AL-RN – 2013) Um típico ciclo de vida de um projeto em TDD consiste em:

- I. Executar os testes novamente e garantir que estes continuem tendo sucesso.
- II. Executar os testes para ver se todos estes testes obtiveram êxito.
- III. Escrever a aplicação a ser testada.
- IV. Refatorar (refactoring).
- V. Executar todos os possíveis testes e ver a aplicação falhar.
- VI. Criar o teste.

A ordem correta e cronológica que deve ser seguida para o ciclo de vida do TDD está expressa em:

- a) IV – III – II – V – I – VI.
- b) V – VI – II – I – III – IV.
- c) VI – V – III – II – IV – I.
- d) III – IV – V – VI – I – II.
- e) III – IV – VI – V – I – II.

Comentários:

A ordem correta é: (VI) Criar o teste; (V) Executar todos os possíveis testes e ver a aplicação falhar; (III) Escrever a aplicação a ser testada; (II) Executar os testes para ver se todos estes testes obtiveram êxito; (IV) Refatorar (refactoring); (I) Executar os testes novamente e garantir que estes continuem tendo sucesso.

Gabarito: Letra C

10. (FGV / ALMT – 2013) Com relação ao desenvolvimento orientado (dirigido) a testes (do Inglês Test Driven Development – TDD), analise as afirmativas a seguir.

- I. TDD é uma técnica de desenvolvimento de software iterativa e incremental.
- II. TDD implica escrever o código de teste antes do código de produção, um teste de cada vez, tendo certeza de que o teste falha antes de escrever o código que irá fazê-lo passar.
- III. TDD é uma técnica específica do processo XP (Extreme Programming), portanto, só pode ser utilizada em modelos de processos ágeis de desenvolvimento de software.

Assinale:

- a) Se somente as afirmativas I e II estiverem corretas.
- b) Se somente as afirmativas I e III estiverem corretas.
- c) Se somente as afirmativas II e III estiverem corretas.
- d) Se somente a afirmativa III estiver correta.
- e) Se somente a afirmativa I estiver correta.

Comentários:

(I) Correto, é iterativo e incremental (lembrando que, em sua imensa maioria, metodologias ágeis são iterativas/incrementais); (II) Correto, escrevem-se os testes antes, fá-lo falhar e só depois escreve-se o código da aplicação; (III) Errado, ele é uma abordagem independente que pode ser utilizado com metodologias ágeis ou tradicionais.

Gabarito: Letra A

11. (FCC / TRT-MG – 2015) Um analista de TI está participando do desenvolvimento de um software orientado a objetos utilizando a plataforma Java. Na abordagem de desenvolvimento adotada, o código é desenvolvido de forma incremental, em conjunto com o teste para esse incremento, de forma que só se passa para o próximo incremento quando o atual passar no teste. Como o código é desenvolvido em incrementos muito pequenos e são executados testes a cada vez que uma funcionalidade é adicionada ou que o programa é refatorado, foi necessário definir um ambiente de testes automatizados utilizando um framework popular que suporta o teste de programas Java.

A abordagem de desenvolvimento adotada e o framework de suporte à criação de testes automatizados são, respectivamente,

- a) Behavior-Driven Development e JTest.

- b) Extreme Programming e Selenium.
- c) Test-Driven Development e Jenkins.
- d) Data-Driven Development and Test e JUnit.
- e) Test-Driven Development e JUnit.

Comentários:

A abordagem claramente é o TDD e o framework evidentemente é o JUnit (ferramenta de suporte à criação de testes unitários automatizados). Lembrando que: DDD é um modelo de programação em que os próprios dados controlam o fluxo do programa e não a lógica do programa; XP é uma metodologia ágil de gerenciamento de projetos que suporta lançamentos frequentes em curtos ciclos de desenvolvimento para melhorar a qualidade do software e permitir que os desenvolvedores respondam às mudanças nos requisitos dos clientes; BDD é um método de desenvolvimento ágil que encoraja a colaboração entre desenvolvedores; Selenium é uma ferramenta usada para testes funcionais em aplicações web; e JTest é um framework de análise estática que usa a linguagem Java.

Gabarito: Letra E

12. (CESPE / TRE-PE – 2017) O desenvolvimento orientado a testes (TDD):

- a) é um conjunto de técnicas que se associam ao XP (extreme programming) para o desenvolvimento incremental do código que se inicia com os testes.
- b) agrega um conjunto de testes de integração para avaliar a interconexão dos componentes do software com as aplicações a ele relacionadas.
- c) avalia o desempenho do desenvolvimento de sistemas verificando se o volume de acessos/transações está acima da média esperada.
- d) averigua se o sistema atende aos requisitos de desempenho verificando se o volume de acessos/transações mantém-se dentro do esperado.
- e) testa o sistema para verificar se ele foi desenvolvido conforme os padrões e a metodologia estabelecidos nos requisitos do projeto.

Comentários:

O único item que faz algum sentido em relação ao TDD é o primeiro, isto é, conjunto de técnicas intimamente ligadas ao XP para o desenvolvimento de software que se inicia com os testes.

Gabarito: Correto

13. (CESPE / STM – 2018) O TDD (test driven development) parte de um caso de teste que caracteriza uma melhoria desejada ou nova funcionalidade a ser desenvolvida, de modo a confirmar o comportamento correto e possibilitar a evolução ou refatoração do código.

Comentários:

TDD é um método para construir software que enfatiza a criação de testes antes da criação do código-fonte. Logo, faz-se o teste - se não passou, refatora!

Gabarito: Correto

14. (CESPE / TRE/PI – 2016) O TDD (test driven development):

a) apresenta como vantagem a leitura das regras de negócio a partir dos testes, e, como desvantagem, a necessidade de mais linhas de códigos que a abordagem tradicional, o que gera um código adicional.

b) impede que seja aplicada a prática de programação em pares, que é substituída pela interação entre analista de teste, testador e programador.

c) é um conjunto de técnicas associadas ao eXtremme Programing e a métodos ágeis, sendo, contudo, incompatível com o Refactoring, haja vista o teste ser escrito antes da codificação.

d) refere-se a uma técnica de programação cujo principal objetivo é escrever um código funcional limpo, a partir de um teste que tenha falhado.

e) refere-se a uma metodologia de testes em que se devem testar condições, loops e operações; no entanto, por questão de simplicidade, não devem ser testados polimorfismos.

Comentários:

(a) Errado, esse item não faz nenhum sentido em relação ao TDD; (b) Errado, não impede a programação em pares; (c) Errado, ela é totalmente compatível e dependente da refatoração; (d) Correto, ela pode ser vista como uma técnica de programação cujo objetivo é escrever um código funcional limpo a partir de um teste falho; (e) Errado, não se trata de uma metodologia de testes.

Gabarito: Letra D

15. (UFRRJ / UFRRJ – 2015) Os testes de unidade têm papel central na metodologia de implementação dirigida por testes, popularizada pelo processo XP e adotada em outros métodos. Esses testes são criados primeiro, exercitando o contrato de cada operação implementada pelos métodos. Em seguida, o código dos métodos é escrito para cumprir os contratos e, portanto, passar nos testes de unidade. Esse cenário corresponde à abordagem

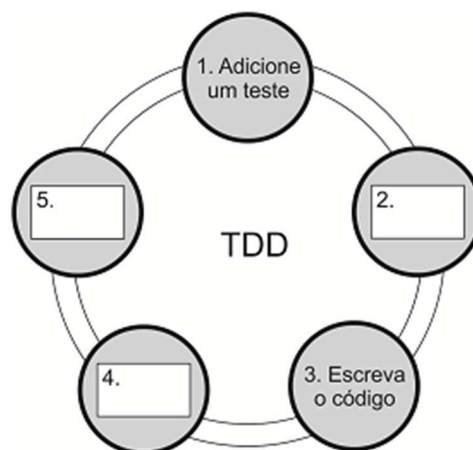
- a) TDD.
- b) MDD.
- c) DDC.
- d) MDE.
- e) FDD.

Comentários:

Testes de unidade têm papel central? Metodologia dirigida a testes? Popularizada pelo XP? Testes são criados primeiro? Tudo isso nos remete a... TDD!

Gabarito: Letra A

16.(FCC / TRE-PR – 2017) Considere o ciclo do Test-Driven Development – TDD.



A Caixa:

- a) 2. corresponde a "Execute os testes automatizados".
- b) 4. corresponde a "Refatore o código".
- c) 5. corresponde a "Execute os testes novamente e observe os resultados".
- d) 4. corresponde a "Execute os testes automatizados".
- e) 5. corresponde a "Faça todos os testes passarem".

Comentários:



(a) Errado, corresponde a "Execute o teste e observe o resultado"; (b) Errado, corresponde a "Execute os testes automatizados"; (c) Errado, corresponde a "Refatore os códigos"; (d) Correto, corresponde realmente a "Execute os testes automatizados"; (e) Errado, corresponde a "Refatore os códigos".

Gabarito: Letra D

17. (IESES / TRE-MA – 2015) A respeito da técnica de testes TDD é correto afirmar que:

- a) Testa o software com base no comportamento esperado.
- b) É uma prática para desenvolvimento de testes unitário que pode utilizar o processo Red-Green-Refactor.
- c) Utiliza-se da estrutura Dado, Quando e Então para montar os testes.
- d) Prega que os testes devem ser realizados sempre após a implementação ser concluída.

Comentários:

(a) Errado, não vejo nada de errado nesse item – ele pode testar o software com base no comportamento esperado! No entanto, a banca considerou o item errado; (b) Correto, ele realmente utiliza o processo RED/GREEN/REFACTOR; (c) Errado, esse item não faz qualquer sentido; (d) Errado, testes são realizados antes de a implementação ser concluída.

Gabarito: Letra B

18. (CESPE / TER-RS – 2015) Projeto para o desenvolvimento de software que utilize TDD deve:

- a) realizar sprints a cada quinzena.
- b) desenvolver pequenos releases.
- c) apresentar grande quantidade de testes unitários de código-fonte previamente desenvolvidos.

- d) apresentar linguagem de programação estruturada.
- e) recomendar a preparação dos testes para que, posteriormente, seja desenvolvido o código.

Comentários:

(a) Errado, não há nenhuma relação entre TDD e Sprints; (b) Errado, são desenvolvidas pequenas unidades e, não, releases; (c) Errado, os testes unitários são escritos iterativamente a medida que o desenvolvimento ocorre; (d) Errado, não há nenhuma relação ou dependência com linguagens de programação; (e) Correto, recomenda-se preparar testes antes do desenvolvimento do código.

Gabarito: Letra E

19.(FCC / TRE-AP – 2015) O TDD – Test Driven Development (Desenvolvimento orientado a teste):

- a) é parte das metodologias ágeis UP – Unified Process e XP – Extreme Programming, tendo sido criado para ser usado em metodologias que respeitam os 4 princípios do Manifesto Ágil.
- b) transforma o desenvolvimento, pois deve-se primeiro implementar o sistema antes de escrever os testes. Os testes são utilizados para facilitar no entendimento do projeto e para clarear o que se deseja em relação ao código.
- c) baseia-se em um ciclo simples: escreve-se um código -> cria-se um teste para passar no código
▪ -> refatora-se.
- d) propõe a criação de testes que validem o código como um todo para reduzir o tempo de desenvolvimento.
- e) beneficia-se de testes que seguem o modelo FIRST: F (Fast) I (Isolated) R (Repeatable) S (Self-verifying) T (Timely).

Comentários:

ETAPA	DESCRIÇÃO
VERMELHO (RED)	Escreva um teste que falha: casos de teste são escritos sem que exista o recurso a ser testado, levando o teste a falhar;
VERDE (GREEN)	Escreva código para passar no teste: o recurso é implementado com o mínimo de código necessário para que passe no teste.
REFATORAR (REFACTOR)	Elimine redundâncias: o código que implementa o recurso é refinado e melhorado, sem que seja adicionada novas funcionalidades.

(a) Errado, Unified Process (UP) não é uma metodologia ágil; (b) Errado, devem ser implementados os testes antes do sistema; (c) Errado, cria-se um teste falho > escreve-se o código que passa > refatora-se; (d) Errado, criam-se testes que validam unidades e, não, o código como um todo; (e) Correto, vamos falar um pouquinho sobre isso agora:

Existe uma representação chamada Modelo FIRST: **F (Fast)**: devem ser rápidos, pois testam apenas uma unidade; **I (Isolated)**: testes unitários são isolados, testando individualmente as unidades e não sua integração; **R (Repeatable)**: repetição nos testes, com resultados de comportamento constante; **S (Self-verifying)**: a auto verificação deve verificar se passou ou se deu como falha o teste; **T (Timely)**: o teste deve ser oportuno, sendo um teste por unidade.

Gabarito: Letra E

20. (CESPE / TRE-TO – 2017) O TDD (Test-Driven Development), que vem sendo adotado para testar os projetos de software,

- a) utiliza os testes de caixa preta antes da entrega do software.
- b) agiliza os testes por amostragem sem compatibilidade retroativa.
- c) cobre amplamente os testes unitários.
- d) escreve o teste antes da codificação do software.
- e) realiza refactoring antes de escrever a aplicação a ser testada.

Comentários:

Eu já começo discordando do enunciado – ele vem sendo utilizado para desenvolver software utilizando testes, mas não para testar projetos de software. Ignorando a redação da questão, vamos aos comentários: (a) Errado, ele tipicamente utiliza testes de unidade antes da entrega do software; (b) Errado, esse item não faz qualquer sentido; (c) Errado, ele não tem o intuito principal de cobrir amplamente testes unitários; (d) Correto, o teste é escrito antes da codificação do software; (e) Errado, não faz sentido fazer *refactoring* antes de escrever a aplicação.

Gabarito: Letra D

LISTA DE QUESTÕES – DIVERSAS BANCAS

1. **(CESPE / INMETRO / 2009)** A rotina diária dos desenvolvedores, ao empregar processos baseados no TDD (Test-Driven Development), é concentrada na elaboração de testes de homologação.
2. **(CESPE / INPI / 2013)** Usando-se o TDD, as funcionalidades devem estar completas e da forma como serão apresentadas aos seus usuários para que possam ser testadas e consideradas corretas.
3. **(CESPE / ANCINE / 2013)** No desenvolvimento de software conforme as diretrizes do TDD (Test-Driven Development), deve-se elaborar primeiramente os testes e, em seguida, escrever o código necessário para passar pelos testes.
4. **(CESPE / INMETRO / 2009)** Considerando uma organização na qual a abordagem de Test Driven Development (TDD) esteja implementada, assinale a opção correta.
 - a) Nessa organização, ocorre a execução de iterações com ciclo longo, isto é, com duração de alguns meses.
 - b) No início de cada iteração, a primeira atividade realizada pela equipe de desenvolvimento é produzir o código que será validado através de testes.
 - c) O refactoring é uma das primeiras atividades realizada no início de cada iteração.
 - d) Entre as atividades finais de cada iteração, o desenvolvedor escreve casos de teste automatizados, cuja execução verifica se houve a melhoria desejada ou se uma nova funcionalidade foi implementada.
 - e) Há coerência e inter-relação com os princípios promovidos pela prática da extreme programming (XP).
5. **(CESPE / MPOG / 2013)** Ao realizar o TDD (test-driven development), o programador é conduzido a pensar em decisões de design antes de pensar em código de implementação, o que cria um maior acoplamento, uma vez que seu objetivo é pensar na lógica e nas responsabilidades de cada classe.
6. **(CESPE / MPU – 2013)** Na metodologia TDD, ou desenvolvimento orientado a testes, cada nova funcionalidade inicia com a criação de um teste, cujo planejamento permite a identificação dos itens e funcionalidades que deverão ser testados, quem são os responsáveis e quais os riscos envolvidos.

7. (CESPE / STF – 2013) No TDD, o primeiro passo do desenvolvedor é criar o teste, denominado teste falho, que retornará um erro, para, posteriormente, desenvolver o código e aprimorar a codificação do sistema.
8. (CESPE / TRT-17 – 2013) TDD consiste em uma técnica de desenvolvimento de software com abordagem embasada em perspectiva evolutiva de seu desenvolvimento. Essa abordagem envolve a produção de versões iniciais de um sistema a partir das quais é possível realizar verificações de suas qualidades antes que ele seja construído.
9. (CESPE / ALRN – 2013) Um típico ciclo de vida de um projeto em TDD consiste em:
- I. Executar os testes novamente e garantir que estes continuem tendo sucesso.
 - II. Executar os testes para ver se todos estes testes obtiveram êxito.
 - III. Escrever a aplicação a ser testada.
 - IV. Refatorar (refactoring).
 - V. Executar todos os possíveis testes e ver a aplicação falhar.
 - VI. Criar o teste.

A ordem correta e cronológica que deve ser seguida para o ciclo de vida do TDD está expressa em:

- a) IV – III – II – V – I – VI.
 - b) V – VI – II – I – III – IV.
 - c) VI – V – III – II – IV – I.
 - d) III – IV – V – VI – I – II.
 - e) III – IV – VI – V – I – II.
10. (FGV / ALMT – 2013) Com relação ao desenvolvimento orientado (dirigido) a testes (do Inglês Test Driven Development – TDD), analise as afirmativas a seguir.

- I. TDD é uma técnica de desenvolvimento de software iterativa e incremental.
- II. TDD implica escrever o código de teste antes do código de produção, um teste de cada vez, tendo certeza de que o teste falha antes de escrever o código que irá fazê-lo passar.
- III. TDD é uma técnica específica do processo XP (Extreme Programming), portanto, só pode ser utilizada em modelos de processos ágeis de desenvolvimento de software.

Assinale:

- a) Se somente as afirmativas I e II estiverem corretas.
- b) Se somente as afirmativas I e III estiverem corretas.
- c) Se somente as afirmativas II e III estiverem corretas.
- d) Se somente a afirmativa III estiver correta.

e) Se somente a afirmativa I estiver correta.

11. (FCC / TRT-MG – 2015) Um analista de TI está participando do desenvolvimento de um software orientado a objetos utilizando a plataforma Java. Na abordagem de desenvolvimento adotada, o código é desenvolvido de forma incremental, em conjunto com o teste para esse incremento, de forma que só se passa para o próximo incremento quando o atual passar no teste. Como o código é desenvolvido em incrementos muito pequenos e são executados testes a cada vez que uma funcionalidade é adicionada ou que o programa é refatorado, foi necessário definir um ambiente de testes automatizados utilizando um framework popular que suporta o teste de programas Java.

A abordagem de desenvolvimento adotada e o framework de suporte à criação de testes automatizados são, respectivamente,

- a) Behavior-Driven Development e JTest.
- b) Extreme Programming e Selenium.
- c) Test-Driven Development e Jenkins.
- d) Data-Driven Development and Test e JUnit.
- e) Test-Driven Development e JUnit.

12. (CESPE / TRE-PE – 2017) O desenvolvimento orientado a testes (TDD):

- a) é um conjunto de técnicas que se associam ao XP (extreme programming) para o desenvolvimento incremental do código que se inicia com os testes.
- b) agrega um conjunto de testes de integração para avaliar a interconexão dos componentes do software com as aplicações a ele relacionadas.
- c) avalia o desempenho do desenvolvimento de sistemas verificando se o volume de acessos/transações está acima da média esperada.
- d) averigua se o sistema atende aos requisitos de desempenho verificando se o volume de acessos/transações mantém-se dentro do esperado.
- e) testa o sistema para verificar se ele foi desenvolvido conforme os padrões e a metodologia estabelecidos nos requisitos do projeto.

13. (CESPE / STM – 2018) O TDD (test driven development) parte de um caso de teste que caracteriza uma melhoria desejada ou nova funcionalidade a ser desenvolvida, de modo a confirmar o comportamento correto e possibilitar a evolução ou refatoração do código.

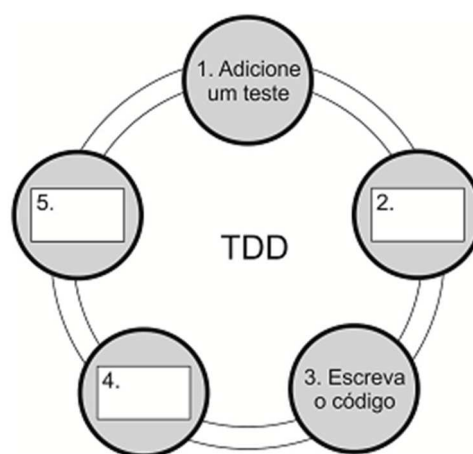
14. (CESPE / TRE/PI – 2016) O TDD (test driven development):

- a) apresenta como vantagem a leitura das regras de negócio a partir dos testes, e, como desvantagem, a necessidade de mais linhas de códigos que a abordagem tradicional, o que gera um código adicional.
- b) impede que seja aplicada a prática de programação em pares, que é substituída pela interação entre analista de teste, testador e programador.
- c) é um conjunto de técnicas associadas ao eXtreme Programming e a métodos ágeis, sendo, contudo, incompatível com o Refactoring, haja vista o teste ser escrito antes da codificação.
- d) refere-se a uma técnica de programação cujo principal objetivo é escrever um código funcional limpo, a partir de um teste que tenha falhado.
- e) refere-se a uma metodologia de testes em que se devem testar condições, loops e operações; no entanto, por questão de simplicidade, não devem ser testados polimorfismos.

15.(UFRRJ / UFRRJ – 2015) Os testes de unidade têm papel central na metodologia de implementação dirigida por testes, popularizada pelo processo XP e adotada em outros métodos. Esses testes são criados primeiro, exercitando o contrato de cada operação implementada pelos métodos. Em seguida, o código dos métodos é escrito para cumprir os contratos e, portanto, passar nos testes de unidade. Esse cenário corresponde à abordagem

- a) TDD.
- b) MDD.
- c) DDC.
- d) MDE.
- e) FDD.

16.(FCC / TRE-PR – 2017) Considere o ciclo do Test-Driven Development – TDD.



A Caixa:

- a) 2. corresponde a "Execute os testes automatizados".
- b) 4. corresponde a "Refatore o código".
- c) 5. corresponde a "Execute os testes novamente e observe os resultados".
- d) 4. corresponde a "Execute os testes automatizados".
- e) 5. corresponde a "Faça todos os testes passarem".

17. (IESES / TRE/MA – 2015) A respeito da técnica de testes TDD é correto afirmar que:

- a) Testa o software com base no comportamento esperado.
- b) É uma prática para desenvolvimento de testes unitário que pode utilizar o processo Red-Green-Refactor.
- c) Utiliza-se da estrutura Dado, Quando e Então para montar os testes.
- d) Prega que os testes devem ser realizados sempre após a implementação ser concluída.

18. (CESPE / TRE/RS – 2015) Projeto para o desenvolvimento de software que utilize TDD deve:

- a) realizar sprints a cada quinzena.
- b) desenvolver pequenos releases.
- c) apresentar grande quantidade de testes unitários de código-fonte previamente desenvolvidos.
- d) apresentar linguagem de programação estruturada.
- e) recomendar a preparação dos testes para que, posteriormente, seja desenvolvido o código.

19. (FCC / TRE/AP – 2015) O TDD – Test Driven Development (Desenvolvimento orientado a teste):

- a) é parte das metodologias ágeis UP – Unified Process e XP – Extreme Programming, tendo sido criado para ser usado em metodologias que respeitam os 4 princípios do Manifesto Ágil.
- b) transforma o desenvolvimento, pois deve-se primeiro implementar o sistema antes de escrever os testes. Os testes são utilizados para facilitar no entendimento do projeto e para clarear o que se deseja em relação ao código.
- c) baseia-se em um ciclo simples: escreve-se um código -> cria-se um teste para passar no código -> refatora-se.
- d) propõe a criação de testes que validem o código como um todo para reduzir o tempo de desenvolvimento.
- e) beneficia-se de testes que seguem o modelo FIRST: F (Fast) I (Isolated) R (Repeatable) S (Self-verifying) T (Timely).

20. (CESPE / TRE/TO – 2017) O TDD (Test-Driven Development), que vem sendo adotado para testar os projetos de software,

- a) utiliza os testes de caixa preta antes da entrega do software.
- b) agiliza os testes por amostragem sem compatibilidade retroativa.
- c) cobre amplamente os testes unitários.
- d) escreve o teste antes da codificação do software.
- e) realiza refactoring antes de escrever a aplicação a ser testada.

GABARITO – DIVERSAS BANCAS

- | | | | |
|-----|---------|-----|---------|
| 1. | ERRADO | 11. | LETRA E |
| 2. | ERRADO | 12. | CORRETO |
| 3. | CORRETO | 13. | CORRETO |
| 4. | LETRA E | 14. | LETRA D |
| 5. | ERRADO | 15. | LETRA A |
| 6. | CORRETO | 16. | LETRA D |
| 7. | CORRETO | 17. | LETRA B |
| 8. | CORRETO | 18. | LETRA E |
| 9. | LETRA C | 19. | LETRA E |
| 10. | LETRA A | 20. | LETRA D |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.